

la5dngduc

May 25, 2026

```
[2]: import fitz
import re
import io
import unicodedata
import pytesseract
import shutil
import pytesseract
```

```
[3]: def nettoyer_bloc(txt):
    return txt.strip()
```

```
[4]: # Supprimer les émojis, les icônes ect.... ( tout caractère bizarre)
```

```
def supprimer_emojis_et_symboles_inutiles(txt):
    resultat = []
    for ch in txt:
        cat = unicodedata.category(ch)

        if ch in "\n\t ":
            resultat.append(ch)
        elif cat.startswith("L"): # lettres
            resultat.append(ch)
        elif cat.startswith("N"): # chiffres
            resultat.append(ch)
        elif cat.startswith("P"): # ponctuation
            resultat.append(ch)
        elif cat.startswith("Z"): # espaces unicode
            resultat.append(" ")
        # sinon on supprime (emoji, icône, pictogramme, etc.)

    return "".join(resultat)
```

```
[5]: def transformer_barres_progression(ligne):
    # élargir les caractères possibles (barres variées)
    m = re.match(r"^(.*?)([ \t#=-\s]{2,})\s*$", ligne)
    if not m:
        return ligne
```

```

competence = m.group(1).strip()
barres = m.group(2)

# caractères considérés comme "vides"
vides = set(" - ")

total = len(barres)
pleins = sum(1 for c in barres if c not in vides)

if total == 0:
    return ligne

# calcul proportionnel (plus précis que juste compter)
niveau = round((pleins / total) * 5)

# sécuriser entre 1 et 5
niveau = max(1, min(niveau, 5))

return f"{competence} niveau {niveau}/5"

```

Cette fonction :

Transforme une barre de progression (ex: , #####—,)

en texte : “compétence niveau X/5”.

- Détecte la barre avec une regex
- Sépare la compétence et la barre
- Calcule le niveau selon la proportion de symboles “pleins”
- Normalise le résultat entre 1 et 5
- Retourne la ligne inchangée si aucune barre n’est trouvée

```

[6]: def nettoyer_cv_final(texte):
    lignes = texte.split("\n")
    nouvelles_lignes = []

    for ligne in lignes:
        ligne = ligne.strip()

        if not ligne:
            nouvelles_lignes.append("")
            continue

```

```

ligne = ligne.replace(" ", "- ")
ligne = ligne.replace("•", "- ")
ligne = ligne.replace(" ", "- ")
ligne = ligne.replace(" ", "- ")
ligne = ligne.replace("→", "- ")
ligne = ligne.replace(" ", "- ")

ligne = transformer_barres_progression(ligne)

if re.fullmatch(r"[-_={4,}]", ligne):
    continue

ligne = supprimer_emojis_et_symboles_inutiles(ligne)
ligne = re.sub(r"[ \t]+", " ", ligne)

nouvelles_lignes.append(ligne)

texte = "\n".join(nouvelles_lignes)
texte = re.sub(r"\n{3,}", "\n\n", texte)

return texte.strip()

```

Elle prend un texte brut et :

nettoie les lignes

uniformise les puces

transforme les barres de progression

supprime le bruit (symboles, emojis, lignes inutiles)

[]:

```

[7]: # Cette fonction prend une ligne structurée (dictionnaire) et renvoie un texte
    ↪ propre.
def texte_ligne_dict(line):
    morceaux = []

    for span in line.get("spans", []):
        t = span.get("text", "")
        if t and t.strip():
            morceaux.append(t.strip())

    return nettoyer_bloc(" ".join(morceaux))

```

```
[8]: def extraire_lignes_page(page):
    # Extrait le contenu d'une page PDF sous forme structurée (PyMuPDF)
    # Chaque ligne est récupérée avec :
    # - son texte nettoyé
    # - ses coordonnées (bbox)
    # - sa largeur et sa hauteur

    data = page.get_text("dict")
    lignes = []

    for block in data.get("blocks", []):
        if block.get("type") != 0:
            continue

        for line in block.get("lines", []):
            x0, y0, x1, y1 = line["bbox"]
            texte = texte_ligne_dict(line)

            if not texte:
                continue

            lignes.append({
                "x0": x0,
                "y0": y0,
                "x1": x1,
                "y1": y1,
                "width": x1 - x0,
                "height": y1 - y0,
                "text": texte
            })

    return lignes
```

```
[ ]:
```

```
[9]: # Détection des colonnes

def clusteriser_colonnes(lignes, largeur_page, gap_x_ratio=0.08):
    if not lignes:
        return []

    lignes = sorted(lignes, key=lambda l: l["x0"])
    gap_x = largeur_page * gap_x_ratio

    clusters = [[lignes[0]]]
```

```

for l in lignes[1:]:
    moyenne_x = sum(x["x0"] for x in clusters[-1]) / len(clusters[-1])

    if abs(l["x0"] - moyenne_x) <= gap_x:
        clusters[-1].append(l)
    else:
        clusters.append([l])

return clusters

```

```

[10]: def overlap_vertical(y0a, y1a, y0b, y1b):
    return max(0, min(y1a, y1b) - max(y0a, y0b))

```

```

[11]: def est_vraie_mise_en_colonnes(clusters):
    # Cette fonction détecte si un texte PDF est organisé en "colonnes"
    # plutôt qu'en une seule colonne.
    #
    # Idée :
    # - On analyse des groupes de lignes (clusters)
    # - On compare leur position horizontale et leur chevauchement vertical
    # - On décide si plusieurs colonnes existent réellement

    if len(clusters) < 2:
        return False

    infos = []
    for cluster in clusters:
        y0 = min(l["y0"] for l in cluster)
        y1 = max(l["y1"] for l in cluster)
        total_chars = sum(len(l["text"]) for l in cluster)
        mean_x = sum(l["x0"] for l in cluster) / len(cluster)

        infos.append({
            "cluster": cluster,
            "y0": y0,
            "y1": y1,
            "total_chars": total_chars,
            "mean_x": mean_x
        })

    infos = sorted(infos, key=lambda x: x["mean_x"])
    gros_clusters = [c for c in infos if c["total_chars"] >= 80]

    if len(gros_clusters) < 2:
        return False

```

```

for i in range(len(gros_clusters)):
    for j in range(i + 1, len(gros_clusters)):
        ov = overlap_vertical(
            gros_clusters[i]["y0"], gros_clusters[i]["y1"],
            gros_clusters[j]["y0"], gros_clusters[j]["y1"]
        )
        if ov >= 80:
            return True

return False

```

```

[12]: def ordonner_clusters(clusters):
    # Cette fonction ordonne des clusters (groupes de lignes PDF)
    # pour reconstruire un ordre de lecture logique.
    #
    # Idée :
    # - Le cluster le plus grand (en nombre de caractères) est supposé être le
    ↪ principal
    # - Les autres clusters sont ordonnés de gauche à droite (mean_x)
    # - On retourne une liste ordonnée de clusters

    infos = []
    for cluster in clusters:
        total_chars = sum(len(l["text"]) for l in cluster)
        mean_x = sum(l["x0"] for l in cluster) / len(cluster)
        infos.append({
            "cluster": cluster,
            "total_chars": total_chars,
            "mean_x": mean_x
        })

    principal = max(infos, key=lambda x: x["total_chars"])
    autres = [c for c in infos if c is not principal]
    autres = sorted(autres, key=lambda x: x["mean_x"])

    return [principal["cluster"]] + [x["cluster"] for x in autres]

```

```

[13]: # Reconstruction fidèle aux lignes
def reconstruire_lignes_ordonnee(lignes_ordonnee, seuil_ligne_vide=18,
    ↪ seuil_meme_ligne=2):
    # Cette fonction reconstruit un texte cohérent à partir de lignes PDF
    ↪ ordonnées
    # en utilisant leurs positions verticales (y0, y1)
    #
    # Idée :
    # - Si deux lignes sont très proches verticalement → même ligne logique
    # - Si elles sont proches → saut de ligne simple

```

```

# - Si elles sont éloignées → paragraphe (ligne vide)

if not lignes_ordonnee:
    return ""

resultat = lignes_ordonnee[0]["text"]
precedente = lignes_ordonnee[0]

for ligne in lignes_ordonnee[1:]:
    gap_y = ligne["y0"] - precedente["y1"]

    # même ligne visuelle → espace
    if abs(ligne["y0"] - precedente["y0"]) < seuil_meme_ligne:
        resultat += " " + ligne["text"]

    # ligne suivante normale → saut de ligne simple
    elif gap_y < seuil_ligne_vide:
        resultat += "\n" + ligne["text"]

    # gros écart vertical → ligne vide
    else:
        resultat += "\n\n" + ligne["text"]

    precedente = ligne

return resultat.strip()

```

```

[14]: def reconstruire_page(page):
    # Cette fonction reconstruit le texte d'une page PDF de manière intelligente
    # en tenant compte de la structure visuelle (colonnes ou non).
    #
    # Étapes :
    # 1. Extraction des lignes avec coordonnées
    # 2. Détection de la structure en colonnes
    # 3. Reconstruction du texte dans le bon ordre de lecture
    # 4. Nettoyage final du texte (CV prêt à analyser)

    largeur_page = page.rect.width
    lignes = extraire_lignes_page(page)

    if not lignes:
        return ""

    clusters = clusteriser_colonnes(lignes, largeur_page)

    # Pas une vraie mise en colonnes → lecture normale haut vers bas
    if not est_vraie_mise_en_colonnes(clusters):

```

```

lignes_ordonnee = sorted(lignes, key=lambda l: (l["y0"], l["x0"]))
texte = reconstruire_lignes_ordonnee(lignes_ordonnee)
return nettoyer_cv_final(texte)

# Vraie mise en colonnes
clusters_ordonnees = ordonner_clusters(clusters)

morceaux = []
for cluster in clusters_ordonnees:
    cluster = sorted(cluster, key=lambda l: (l["y0"], l["x0"]))
    morceaux.append(reconstruire_lignes_ordonnee(cluster))

texte = "\n".join(m for m in morceaux if m.strip())
return nettoyer_cv_final(texte)

```

```

[15]: def ocr_page(page, lang="fra+eng"):
    # Cette fonction applique de l'OCR (reconnaissance de texte) sur une page
    ↪ PDF
    # afin d'extraire le texte même si le PDF est une image (scan).
    #
    # Étapes :
    # 1. Vérifie que les bibliothèques OCR sont disponibles
    # 2. Convertit la page PDF en image haute résolution
    # 3. Applique Tesseract OCR pour extraire le texte
    # 4. Nettoie le texte obtenu pour un usage propre (CV, analyse, etc.)

    try:
        import pytesseract
        from PIL import Image
    except Exception:
        return ""

    pix = page.get_pixmap(matrix=fitz.Matrix(2, 2), alpha=False)
    img_bytes = pix.tobytes("png")
    image = Image.open(io.BytesIO(img_bytes))

    texte = pytesseract.image_to_string(image, lang=lang)
    return nettoyer_cv_final(texte)

```

```
[ ]:
```

```

[16]: # Extraction complète d'un CV PDF
def extraire_texte_cv(pdf_path, activer_ocr=True, seuil_min_caracteres=80):
    doc = fitz.open(pdf_path)
    pages_textes = []

    for page in doc:

```



```

texte_page = reconstruire_page(page)

if activer_ocr and len(texte_page.strip()) < seuil_min_caracteres:
    texte_ocr = ocr_page(page)
    if len(texte_ocr.strip()) > len(texte_page.strip()):
        texte_page = texte_ocr

if texte_page.strip():
    pages_textes.append(texte_page.strip())

texte_final = "\n\n==== PAGE SUIVANTE =====\n\n".join(pages_textes)
texte_final = nettoyer_cv_final(texte_final)

return texte_final

```

```

[17]: # Sauvegarde

def sauvegarder_texte(texte, chemin_sortie):
    with open(chemin_sortie, "w", encoding="utf-8") as f:
        f.write(texte)

```

```

[18]: # Test
texte = extraire_texte_cv("PDF_TO_TEXT/cvs/cv_01.pdf", activer_ocr=True)
print(texte)

```

Sophie Marchand
NLP Engineer Senior
I IA · NLP · Embeddings · Semantic Search

I FORMATION

Master Intelligence Artificielle, Université Paris-Saclay, 2019
Spécialisation Traitement Automatique des Langues
Licence Informatique, Université Paris-Sud, 2017

I EXPÉRIENCE

NLP Engineer Senior, Doctrine.fr Paris
I 2021 à aujourd'hui
I Développement de pipelines NLP pour l'analyse juridique avec Python et spaCy
I Implémentation de systèmes de recherche sémantique avec embeddings et similarité cosinus
I Fine-tuning de modèles BERT et RoBERTa sur des corpus juridiques
I Déploiement d'APIs FastAPI en production, monitoring via Prometheus
I Gestion du code avec Git, GitHub, CI/CD GitHub Actions
I Utilisation de FAISS pour la recherche vectorielle à grande échelle

NLP Engineer, Qwant Paris

I 2019-2021

I Développement de modèles de classification de texte avec HuggingFace

I Création d'embeddings multilingues avec sentence-transformers
I RAG pour un système de questions-réponses interne
SM

u sophie.marchand@email.fr
I Paris
I 06 11 22 33 44
I COMPÉTENCES
Python

HuggingFace

spaCy

NLTK

sentence-transformers

FAISS

similarité cosinus

embeddings

BERT

RoBERTa

LLMs

RAG

FastAPI

Docker

Git

CI/CD

PostgreSQL

I LANGUES
Français
Anglais
Natif
C1

[]:

[]:

[]: